TALLER III
IS924 ACS
ING. OSCAR ANDRES SIERRA JARAMILLO

Objetivo General

Evaluar la capacidad del estudiante para **analizar, diseñar e implementar soluciones concurrentes** utilizando primitivas de sincronización dentro de escenarios de **arquitectura cliente-servidor**.

Competencias a Desarrollar

- Diferenciar **procesos vs hilos** en sistemas concurrentes.
- Analizar problemas de **concurrencia y paralelismo**.
- Identificar **condiciones de carrera, deadlocks y starvation**.
- Implementar sincronización usando:
  - ✓ Locks
  - ✓ RLocks
  - ✓ Semáforos
  - ✓ Variables condicionales
  - ✓ Eventos
  - ✓ Barreras
  - ✓ Temporizadores
- Diseñar soluciones concurrentes para **sistemas cliente-servidor**.

SECCIÓN 1: Conceptual.

Responda a las siguientes preguntas de manera clara, concisa y técnicamente precisa.

1. **(5 puntos)** Explique la diferencia fundamental entre **concurrencia** y **paralelismo**. Proporcione un ejemplo de la vida cotidiana (no informático) para cada concepto que ilustre claramente esta diferencia.
2. **(5 puntos)** Defina qué es una **condición de carrera (race condition)**. En el siguiente fragmento de código en Python, identifique la condición de carrera y explique por qué ocurre.
3. **(5 puntos)** Un **deadlock** (bloqueo mutuo) y la **inanición (starvation)** son dos problemas graves en sistemas concurrentes.

Compare y contraste estos dos conceptos. ¿Cuál de los dos considera más fácil de prevenir en el diseño de un sistema y por qué?

4. **(5 puntos)** En el contexto de un servidor que atiende a múltiples clientes, describa una situación donde el uso de **múltiples procesos** (vía multiprocessing) sería más ventajoso que el uso de **múltiples hilos** (vía threading). Mencione al menos dos ventajas y una desventaja de usar procesos en este escenario.

SECCIÓN 2: Análisis de Código (20%)

Analice los siguientes fragmentos de código. Para cada uno, identifique **todos** los problemas de concurrencia presentes, explique los riesgos de ejecución y proponga una corrección concreta (puede describirla o escribir el código corregido).

1. El Cajero Automático
   Problemas identificados:
   - Riesgos:
   - Propuesta de corrección:

2. El Contador Compartido (con doble verificación)
   Problemas identificados:
   - Riesgos:
   - Propuesta de corrección:

SECCIÓN 3: Ejercicios de Programación (40%)

Implemente las soluciones a los siguientes problemas en Python. Su código debe ser claro, eficiente y estar libre de condiciones de carrera.

TALLER III
IS924 ACS
ING. OSCAR ANDRES SIERRA JARAMILLO

1. **Productor-Consumidor con Almacén Limitado (Bounded Buffer)**

   Contexto: Un sistema de procesamiento de imágenes tiene un buffer compartido de tamaño fijo N=10. Un hilo productor genera "tareas de imagen" (puede simularse con un número entero) y las coloca en el buffer. Múltiples hilos consumidores (2) toman las tareas del buffer para procesarlas (simular el procesamiento con time.sleep de duración aleatoria).

   Requisitos:
   - El productor no debe poder añadir una tarea si el buffer está lleno.
   - Los consumidores no deben poder tomar una tarea si el buffer está vacío.
   - El acceso al buffer debe ser seguro para evitar corrupción de datos.
   - Utilice threading.Semaphore (o Condition) para la sincronización.

   Tareas:
   1. Implemente la clase BufferCompartido que gestione la cola y la sincronización.
   2. Implemente la función productor que genere 20 tareas.
   3. Implemente la función consumidor que procese tareas hasta que se acaben.
   4. Asegúrese de que el programa termine correctamente una vez que todas las tareas hayan sido producidas y consumidas.

Ejemplo de Salida Esperada:
text
Productor: Tarea 1 añadida. Buffer: [1]
Productor: Tarea 2 añadida. Buffer: [1, 2]
Consumidor-1: Tomó tarea 1. Buffer: [2]
Consumidor-2: Tomó tarea 2. Buffer: []
Productor: Tarea 3 añadida. Buffer: [3]
...
Consumidor-2: Procesó tarea 20. Buffer: []
Todas las tareas fueron procesadas.

2. **Simulación de Carrera de Autos con Barrera**

Contexto: Simule una carrera de 5 autos (hilos). Cada auto tarda un tiempo diferente en "llegar a la pista" (simulado con time.sleep aleatorio). Una vez que todos los autos han llegado a la línea de salida, la carrera comienza simultáneamente para todos.

Requisitos:

- Utilice threading.Barrier para sincronizar a los 5 autos en la línea de salida.
- Cada auto (hilo) debe imprimir un mensaje cuando "llega a la salida" y esperar en la barrera.
- Después de que la barrera se libere, cada auto debe imprimir un mensaje indicando que la carrera ha comenzado para él.

Tareas:

1. Implemente la función auto que reciba un identificador y una barrera.
2. Cree una barrera para 5 hilos.
3. Inicie 5 hilos, cada uno simulando un auto.

Ejemplo de Salida Esperada:

text

```
Auto 1 llegó a la salida y está esperando.
Auto 3 llegó a la salida y está esperando.
Auto 2 llegó a la salida y está esperando.
Auto 5 llegó a la salida y está esperando.
Auto 4 llegó a la salida y está esperando.
--- ¡CARRERA! ---
Auto 4 inició la carrera.
Auto 1 inició la carrera.
Auto 3 inició la carrera.
Auto 5 inició la carrera.
Auto 2 inició la carrera.
```

3. **Temporizador con Timeout en una Operación**

Context: Un módulo de un sistema de monitoreo intenta conectarse a un servicio externo. Si la conexión tarda más de 2 segundos, debe abortar el intento y registrar un error. La conexión se simula con una función conectar_a_servicio() que puede tardar entre 1 y 5 segundos aleatoriamente.

Requisitos:

- Utilice threading.Timer para implementar el timeout.
- El temporizador debe ejecutar una función que cancele la operación principal (estableciendo una bandera de evento, por ejemplo) si el tiempo se agota.
- El hilo principal debe verificar si la conexión fue exitosa o si el temporizador la canceló.

Tareas:

1. Implemente la función conectar_a_servicio() que usa time.sleep(random.randint(1,5)) y luego retorna "Conectado".
2. Implemente la función timeout_expirado(evento_cancelacion) que imprime un mensaje y activa el evento.
3. En el main, cree un evento de cancelación, inicie un Timer de 2 segundos, y luego ejecute la conexión. Use event.wait(timeout=...) o verifique el estado del evento para determinar si la conexión tuvo éxito o no.

Ejemplo de Salida Esperada:

Caso 1 (Éxito): Conexión exitosa: Conectado.

Caso 2 (Timeout): Timeout: La conexión tardó demasiado. Operación cancelada.

SECCIÓN 4: Problema Aplicado

Sistema de Procesamiento de Vídeos con Prioridades

Usted está diseñando el núcleo de un sistema de transcodificación de vídeo en la nube. El sistema recibe solicitudes de clientes para convertir vídeos a diferentes formatos. Los clientes tienen diferentes niveles de servicio: **Premium** y **Gratuito**.

- **Arquitectura:** Múltiples clientes (hilos) envían trabajos a un servidor. El servidor mantiene una **cola de trabajos** compartida. Un conjunto de **workers (hilos de procesamiento)** toman los trabajos de la cola y los procesan (simular el procesamiento con time.sleep).
- **El Problema:** Los clientes **Premium** deben tener prioridad sobre los clientes **Gratuitos**. Un trabajo Premium que llega debe ser colocado al frente de la cola (o ser el próximo en ser procesado, incluso si hay trabajos Gratuitos esperando). Sin embargo, no se debe permitir que los trabajos Gratuitos mueran de inanición (*starvation*); eventualmente deben ser procesados.

- **Especificaciones:**

    1. **Cola con Prioridad:** Debe implementar una cola que soporte dos niveles de prioridad (Premium > Gratuito).
    2. **Sincronización:** El acceso a la cola de trabajos debe ser thread-safe. Los workers deben esperar si no hay trabajos.
    3. **Manejo de la Inanición:** Diseñe un mecanismo simple para garantizar que los trabajos Gratuitos no sean puestos en espera indefinidamente. Por ejemplo, después de procesar un número determinado de trabajos Premium consecutivos, el sistema debe forzar el procesamiento de un trabajo Gratuito si existe. Usted decide el mecanismo.
    4. **Generación de Trabajos:** Cree 3 clientes Premium y 5 clientes Gratuitos. Cada cliente enviará entre 5 y 10 trabajos. Los clientes deben enviar trabajos de forma concurrente.

Se pide:

1. **Diseño:** Dibuje un diagrama de la arquitectura propuesta (clientes,

cola, workers) y justifique brevemente las primitivas de sincronización que planea utilizar (Locks, Semáforos, Conditions, Colas thread-safe de Python) y cómo implementará la prioridad sin causar inanición.

2. **Implementación:** Escriba el código que implemente su diseño. Debe quedar claro en la salida del programa qué tipo de cliente envió el trabajo y cómo se respeta la prioridad. ***(SE DEBE APLICAR LOS PATRONES DE DISEÑO)***.

Ejemplo de Salida Esperada (simplificada):

text
Worker-1: Procesando trabajo [VIDEO-A] de Cliente-Premium-1
Worker-2: Procesando trabajo [VIDEO-B] de Cliente-Premium-2
Cliente-Gratis-1: Envió trabajo [VIDEO-C]
Worker-1: Procesando trabajo [VIDEO-C] de Cliente-Gratis-1 (Se procesa uno gratis después de algunos premiums)
Worker-2: Procesando trabajo [VIDEO-D] de Cliente-Premium-3
...
Cliente-Gratis-2: Envió trabajo [VIDEO-E]
Worker-1: Procesando trabajo [VIDEO-E] de Cliente-Gratis-2
...
--- Sistema finalizado ---